

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Інженерно-хімічний факультет
Кафедра «Технічні та програмні засоби автоматизації»

Розрахункова робота
Програмування-2. Об'єктно-орієнтоване програмування

Варіант 13
«Інформаційна система Фітнесцентр»

Виконала: студентка гр. ЛА-11, Костів Я.І.

Перевірів: к.т.н., доцент Д.О. Ковалюк

Київ 2022

ЗМІСТ

1. Технічне завдання.
2. Розділ 1 – Опис предметної області.
3. Розділ 2 – Структура програмного забезпечення.
4. Висновки.
5. Додатки.

Технічне завдання

Реалізувати інформаційну систему фітнесцентру, що виконує наступні функції:

- Сортуння клієнтів за прізвищем,
- Сортуння клієнтів за тривалістю відвідування,
- Пошук клієнтів, які ходять в басейн,
- Додавання, редагування, видалення клієнтів.

Розділ 1

Сутність «Клієнт» - атрибути:

- Прізвище клієнта – строка,
- Місто куди направляється листопад,
- тип абонементу – фіксований набір значень

(вір, басейн, тренажерний зал),

- Кількість років, які займається клієнт - ціле число.

Розділ 2

Class Customer - містить інформацію про клієнта

private int id	Ідентифікаційний номер
private String name	Ім'я
private int years	Кількість років, які займається клієнт
private Type subscription	Тип абонементу

Class FitnessCentre – менеджер роботи з листом

public void loadCustomers public void saveCustomers	Завантаження і збереження клієнтів
public void addCustomer	Додавання клієнта
public Customer findCustomerByPool	Пошук клієнтів які ходять в басейн
public boolean deleteCustomer	Видалення клієнта
public void editCustomer	Редагування клієнта
public static void sortCustomers	Сортування клієнтів

```
public class CustomersIDComparator implements  
Comparator<Customer>,  
public class CustomersNameComparator implements  
Comparator<Customer>,  
public class CustomersYearsComparator implements  
Comparator<Customer> - це класи для зрівняння параметрів
```

```
File Edit View Navigate Code Refactor Build Run Tools VCS Window Help FitnessCentre [C:\Users\Super Yanka\my-app\FitnessCentre] - FitnessCentre.java
FitnessCentre src main java FitnessCentre sortCustomers
Run: App
-----Fitness Centre-----
----- True Body Positive -----

(1) - список клієнтів (2) - за алфавітом
(3) - за тривалістю відвідування(у роках)
(4) - клієнти, які ходять в басейн
(5) - додати клієнта (6) - редагувати
(7) - видалити (0) - Quit

Виберіть пункт меню:
1
~Список клієнтів~
|Номер|Прізвище|Тривалість відвідування(у роках)|Тип абонементу|
| 1 |Mark| 1 | POOL |
| 2 |Alina| 1 | GYM |
| 3 |Nastya| 2 | GYM |
| 4 |Tanya| 4 | GYM |
| 5 |Alice| 6 | VIP |

(1) - список клієнтів (2) - за алфавітом
(3) - за тривалістю відвідування(у роках)
(4) - клієнти, які ходять в басейн
(5) - додати клієнта (6) - редагувати
(7) - видалити (0) - Quit

Виберіть пункт меню:
```

```
File Edit View Navigate Code Refactor Build Run Tools VCS Window Help FitnessCentre [C:\Users\Super Yanka\my-app\FitnessCentre] - FitnessCentre.java
FitnessCentre src main java FitnessCentre sortCustomers
Run: App FitnessCentre.java FCManager.java Customer.java
Run: App
2
~Клієнти за алфавітом~
|Номер|Прізвище|Тривалість відвідування(у роках)|Тип абонементу|
| 5 |Alice | 6 | VIP |
| 2 |Alina | 1 | GYM |
| 1 |Mark | 1 | POOL |
| 3 |Nastya | 2 | GYM |
| 4 |Tanya | 4 | GYM |

(1) - список клієнтів (2) - за алфавітом
(3) - за тривалістю відвідування(у роках)
(4) - клієнти, які ходять в басейн
(5) - додати клієнта (6) - редагувати
(7) - видалити (0) - Quit

Виберіть пункт меню:
3

~Клієнти за роком відвідування~
|Номер|Прізвище|Тривалість відвідування(у роках)|Тип абонементу|
| 1 |Mark | 1 | POOL |
| 2 |Alina | 1 | GYM |
| 3 |Nastya | 2 | GYM |
| 4 |Tanya | 4 | GYM |
| 5 |Alice | 6 | VIP |

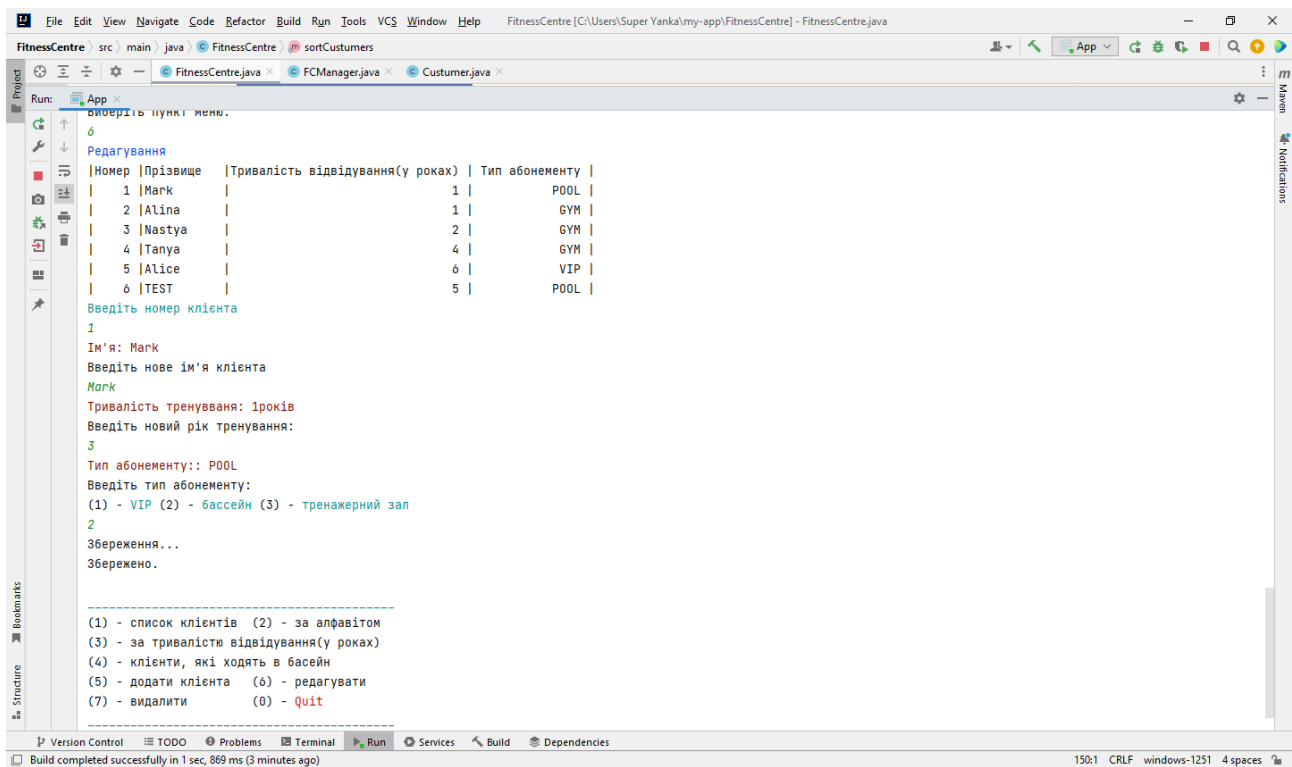
(1) - список клієнтів (2) - за алфавітом
(3) - за тривалістю відвідування(у роках)
(4) - клієнти, які ходять в басейн
(5) - додати клієнта (6) - редагувати
```

```
File Edit View Navigate Code Refactor Build Run Tools VCS Window Help FitnessCentre [C:\Users\Super Yanka\my-app\FitnessCentre] - FitnessCentre.java
FitnessCentre src main java FitnessCentre sortCustomers
Run: App FitnessCentre.java FCManager.java Customer.java
Run: App
Виберіть пункт меню.
4
Клієнти, які ходять в басейн:
Mark тренується в басейні

(1) - список клієнтів (2) - за алфавітом
(3) - за тривалістю відвідування(у роках)
(4) - клієнти, які ходять в басейн
(5) - додати клієнта (6) - редагувати
(7) - видалити (0) - Quit

Виберіть пункт меню:
5
Додати клієнта
Введіть ім'я клієнта:
TEST
Введіть тривалість відвідування клієнта:
5
Введіть тип абонементу:
(1) - VIP (2) - басейн (3) - тренажерний зал
2
Збереження...
Збережено.

(1) - список клієнтів (2) - за алфавітом
(3) - за тривалістю відвідування(у роках)
(4) - клієнти, які ходять в басейн
(5) - додати клієнта (6) - редагувати
(7) - видалити (0) - Quit
```



Висновок

В даній роботі:

1. Реалізовано роботу в інтерактивному режимі з використанням меню
2. Реалізовано завантаження даних з файлу Excel на основі інтерфейсу
3. Реалізовано роботу з колекціями, зокрема операції додавання, редагування, пошук, сортування.

Додаток – Лістинг програми

```
public class Customer implements Comparable<Customer>{

    private int id;

    // прізвище клієнта - строка
    private String name;
    //кількість років, які займається клієнт - ціле число
    private int years;
    // тип абонементу - фіксований набір значень (vip, басейн, тренажерний зал)
    private Type subscription;

    private static int numberOfCustomers = 1;
    public Customer(String name, int years) {
        this.name = name;
        this.years = years;
        id = numberOfCustomers++;
    }

    public int getId() {
        return id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getYears() {
        return years;
    }
    public void setYears(int years) {
        this.years = years;
    }
    public Type getSubscription() {return subscription;}
    public void setSubscription(Type subscription) {this.subscription =
subscription;}

    @Override
    public int compareTo(Customer anotherCustomer) {
        if(this.years == anotherCustomer.years ){
            return -1;
        }else {
```

```

        return 1;
    }
}

```

```

enum Type{
    VIP,
    POOL,
    GYM
}

```

```

import org.apache.xmlbeans.SchemaType;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Iterator;
import java.util.List;

```

```

public class FitnessCentre {

```

```

    // сортування клієнтів за прізвищем
    public static final int SORT_BY_NAME = 1;
    // сортування клієнтів за тривалістю відвідування
    public static final int SORT_BY_YEARS = 2;
    public static final int SORT_BY_ID = 3;

```

```

    private String title;
    static List<Customer> customers;

```

```

    private DataSource customersDataSource;

```

```

    public FitnessCentre(String title) {
        this.title = title;
    }

```

```

    public void loadCustomers(int sourceType, String fileName){
        switch(sourceType) {
            case DataSource.SOURCE_CSV:
                customersDataSource = new CSVLoader();
                customers = customersDataSource.loadCustomers(fileName);
                break;

            case DataSource.SOURCE_EXCEL:
                customersDataSource = new ExcelLoader();

```

```

        customers = customersDataSource.loadCustomers(fileName);
        break;

    case DataSource.SOURCE_GENERATE:
        customers = new ArrayList<>();

        try{
            Customer currentCustomer = new Customer("XXX", 0);
            currentCustomer.setSubscription(Type.VIP);
            customers.add(currentCustomer);

            customers.add(new Customer("Alina", 3));
            customers.get(1).setSubscription(Type.GYM);
            customers.add(new Customer("Tanya", 4));
            customers.get(2).setSubscription(Type.POOL);
            customers.add(new Customer("Alice", 6));
            customers.get(3).setSubscription(Type.VIP);
            customers.add(new Customer("Nastya", 2));
            customers.get(4).setSubscription(Type.GYM);
        } catch (Exception e) {
            throw new RuntimeException(e);
        } break;
    }
}

public void saveCustomers(int sourceType, String sourceName){
    switch(sourceType){
        case DataSource.SOURCE_CSV:
            customersDataSource = new CSVLoader();
            customersDataSource.saveCustomers(sourceName, customers);
            break;

        case DataSource.SOURCE_EXCEL:
            customersDataSource = new ExcelLoader();
            customersDataSource.saveCustomers(sourceName, customers);
            break;
    }
}

public void addCustomer(Customer c){
    customers.add(c);
}

public Customer findCustomerByPool(SchemaType type){
    for (Customer pool : customers){
        switch (pool.getSubscription()){

```

```

        case P00L:
            System.out.println(pool.getName() + " " + " тренується в
басейні");
            break;
        }
    }
    return null;
}

```

```

public boolean deleteCustomer(int customerID){
    for (Iterator<Customer> it = customers.iterator(); it.hasNext();) {
        Customer currentCustomer = it.next();
        if (currentCustomer.getId() == customerID) {
            it.remove();
            return true;
        }
    }
    return false;
}

```

```

public Customer findCustomerByID(int id){
    for (Customer customer : customers){
        if (customer.getId() == id){
            return customer;
        }
    }
    return null;
}

```

```

public void editCustomer(Customer c){
    for (Iterator<Customer> it = customers.iterator(); it.hasNext();){
        Customer currentCustomer = it.next();
        if(currentCustomer.getId() == c.getId()){
            currentCustomer.setName(c.getName());
            currentCustomer.setYears(c.getYears());
            currentCustomer.setSubscription(c.getSubscription());
        }
    }
}

```

```

public static void sortCustomers(int orderID){
    switch(orderID){
        case SORT_BY_NAME:
            Collections.sort(customers, new CustomersNameComparator());
            break;
        case SORT_BY_YEARS:
            Collections.sort(customers, new CustomersYearsComparator());
            break;
        case SORT_BY_ID:

```

```

        Collections.sort(customers, new CustomersIDComparator());
        break;
    }
}

public static void printCustomers(){
    System.out.format("|%5s |%-10s |%10s |%15s |\n",
        "Номер", "Прізвище", "Тривалість відвідування(у роках)", "Тип
абонементу");

    for (Customer currentCustomer : customers) {
        System.out.format("|%5d |%-10s |%32d |%15s |\n",
            currentCustomer.getId(), currentCustomer.getName(),
            currentCustomer.getYears(),
currentCustomer.getSubscription().name());
        }
    }
}

import java.util.Comparator;

public class CustomersNameComparator implements Comparator<Customer> {

    @Override
    public int compare(Customer c1, Customer c2) {
        int compareName = c1.getName().compareTo(c2.getName());
        if(compareName != 0) return compareName;
        return c2.getYears()- c1.getYears();

    }
}

import java.util.Comparator;

public class CustomersYearsComparator implements Comparator<Customer> {

    @Override
    public int compare(Customer c1, Customer c2) {
        int yearsCompate = Double.compare(c1.getYears(), c2.getYears());
        if(yearsCompate == 0){
            return c2.getName().compareTo(c1.getName());
        } else {
            return yearsCompate;
        }

    }
}

```

```

}
import java.util.Comparator;

public class CustomersIDComparator implements Comparator<Customer> {

    @Override
    public int compare(Customer c1, Customer c2) {
        int IDCompare = Double.compare(c2.getId(), c1.getId());
        if (IDCompare == 0) {
            return c1.getName().compareTo(c2.getName());
        } else {
            return c1.getId()-c2.getId();
        }
    }
}

```

```
import jdk.jshell.JShell;
```

```
import java.util.Scanner;
```

```
import static
```

```
org.openxmlformats.schemas.drawingml.x2006.main.CTFillProperties.type;
```

```
public class FCManager {
```

```
    private Scanner in = new Scanner(System.in);
```

```
    private FitnessCentre FC = new FitnessCentre("FC");
```

```
    public void printMenu(){
```

```
        System.out.println();
```

```
        System.out.println ((char) 27 +
```

```
"[36m-----" + (char)27 + "[0m");
```

```
        System.out.println(
```

```
            "(1) - список клієнтів " +
```

```
            "(2) - за алфавітом \n(3) - за тривалістю відвідування(у
```

```
роках)\n(4) - клієнти, " +
```

```
            "які ходять в басейн \n(5) - додати клієнта (6) -
```

```
редагувати \n(7) - " +
```

```
            "видалити " + "
```

```
            (0) -" + (char) 27 + "[31m Quit" +
```

```
(char)27 + "[0m");
```

```
        System.out.println ((char) 27 +
```



```

"[36m-----" + (char)27 + "[0m");
    System.out.println();

}

public void selectMenu(){

    int fileType = DataSource.SOURCE_EXCEL;
    String sourceName = " --- ";

    FC.loadCustomers(fileType, sourceName);

    boolean quit = false;
    int menuItem;

    do {
        System.out.println("Виберіть пункт меню: ");
        menuItem = in.nextInt();

        switch (menuItem) {
            case 1:
                System.out.println((char)27 + "[34m
~Список клієнтів~ " + (char)27 + "[0m");
                FC.sortCustomers(FitnessCentre.SORT_BY_ID);
                FC.printCustomers();
                break;
            case 2:
                System.out.println((char)27 + "[34m
~Клієнти за алфавітом~ " + (char)27 + "[0m");
                FC.sortCustomers(FitnessCentre.SORT_BY_NAME);
                FC.printCustomers();
                break;
            case 3:
                System.out.println((char)27 + "[34m
~Клієнти за роком відвідування~ " + (char)27 + "[0m");
                FC.sortCustomers(FitnessCentre.SORT_BY_YEARS);
                FC.printCustomers();
                break;
            case 4:
                System.out.println((char)27 + "[35mКлієнти, які ходять в
басейн:" + (char)27 + "[0m");
                findCustomer();
                break;
            case 5:
                System.out.println((char)27 + "[34mДодати клієнта " + (char)27
+ "[0m");

```

```

        Customer c = enterCustomer();
        FC.saveCustomers(fileType, sourceName);
        FC.addCustomer(c);
        break;
    case 6:
        System.out.println((char)27 + "[34mРедагування " + (char)27 +
"[0m");

        FC.printCustomers();
        editCustomer();
        FC.saveCustomers(fileType, sourceName);
        break;
    case 7:
        System.out.println((char)27 + "[34mВидалення" + (char)27 +
"[0m");

        FC.printCustomers();
        deleteCustomer();
        FC.saveCustomers(fileType, sourceName);
        break;
    case 0:
        quit = true;
        FC.saveCustomers(fileType, sourceName);
        break;
    default:
        System.out.println((char)27 + "[31mInvalid choice." + (char)27
+ "[0m");
    }
    if(!quit){
        printMenu();
    }
} while (!quit);
System.out.println((char)27 + "[36mBye-bye!" + (char)27 + "[0m");
}

private void findCustomer() {
    FC.findCustomerByPool(type);
}

private Customer enterCustomer() {
    System.out.println("Введіть ім'я клієнта: ");
    String name = in.next();

    System.out.println("Введіть тривалість відвідування клієнта: ");
    int years = in.nextInt();

    System.out.println("Введіть тип абонементу: " );
    System.out.println( "(1) - " + (char)27 + "[36mVIP " +

```

```
        ""+ (char)27 + "[0m" + "(2) - " + (char)27 + "[36mbассейн " +
        ""+ (char)27 + "[0m" + "(3) - " + (char)27 + "[36mтренажерний зал
"+ (char)27 + "[0m" );
```

```
int type = in.nextInt();
```

```
    Type customerType;
    switch(type){
        case 1:
            customerType = Type.VIP;
            break;
        case 2:
            customerType = Type.POOL;
            break;
        case 3:
            customerType = Type.GYM;
            break;
        default:
            customerType = Type.VIP;
    }
    Customer c = new Customer(name, years);
    c.setSubscription(customerType);
    return c;
}
```

```
private void deleteCustomer() {
    System.out.println ((char) 27 + "[36mВведіть номер клієнта" + (char)27 +
"[0m");
    int customerID = in.nextInt();

    Customer c = FC.findCustomerByID(customerID);
    if(c == null){
        System.err.println((char)27 + "[31mCustomer not found"+ (char)27 +
"[0m");
    } else {
        FC.deleteCustomer(customerID);
    }
}
```

```
private void editCustomer(){
    System.out.println ((char) 27 + "[36mВведіть номер клієнта" + (char)27 +
"[0m");
    int customerID = in.nextInt();
    Customer c = FC.findCustomerByID(customerID);
    if(c == null){
```

```

        System.err.println((char)27 + "[31mCustomer not found" + (char)27 +
"[0m");
    } else {
        System.err.println("Ім'я: " + c.getName());
        System.out.println("Введіть нове ім'я клієнта");
        String name = in.next();
        if(!name.isEmpty()){
            c.setName(name);
        } else if(name.isEmpty()){
            c.getName();
        }
        System.err.println("Тривалість тренування: " + c.getYears() +
"років");
        System.out.println("Введіть новий рік тренування:");
        int years = in.nextInt();
        if(years != 0){
            c.setYears(years);
        }
        System.err.println("Тип абонементу:: " + c.getSubscription());
        System.out.println("Введіть тип абонементу: ");
        System.out.println( "(1) - " + (char)27 + "[36mVIP " +
            "" + (char)27 + "[0m" + "(2) - " + (char)27 + "[36mbасейн " +
            "" + (char)27 + "[0m" + "(3) - " + (char)27 + "[36mтренажерний
зал " + (char)27 + "[0m" );
        int type = in.nextInt();

        Type customerType;
        switch(type){
            case 1:
                customerType = Type.VIP;
                break;
            case 2: customerType = Type.POOL;
                break;
            case 3: customerType = Type.GYM;
                break;
            default:
                customerType = Type.VIP;
        }
        c.setSubscription(customerType);
        FC.editCustomer(c);
    }
}
}

```